ChatGPT

EnergyMind Tunisia: ML and Physics-Based Approaches for Power Systems

Executive Summary

This report surveys recent machine-learning approaches (mainly from top AI conferences) relevant to power systems, and analyzes their applicability to the EnergyMind Tunisia challenge (energy resilience, state estimation, anomaly/attack detection, etc.). Key findings: spatio-temporal graph models (e.g. Graph WaveNet, DCRNN) can capture network-dependent dynamics; advanced time-series forecasters (Transformers, N-BEATS) provide strong prediction baselines; physics-informed networks (PINNs) can embed power-flow laws; and hybrid pipelines combining classical state estimators with ML detectors are promising. For each approach we summarize its method, data, and trade-offs, then discuss required adaptations (e.g. grid topology, PMU data) and risks (adversarial attacks). We propose several concrete baseline pipelines (in tabular form) blending physics and ML, and outline a Phase 1 prototype plan (with simulators like MATPOWER/PandaPower, IEEE test cases, fault injection) and team roles. Finally we prioritize methods to implement first (favoring proven spatio-temporal models with available code【37†L31-L39】【44†L23-L31】) and recommend evaluation metrics.

Recent ML Approaches (Last ~5 Years)

- **Graph WaveNet (Wu *et al.*, IJCAI 2019)** – A spatial-temporal graph neural net with *adaptive adjacency* and *dilated 1D convolutions*【37†L31-L39】. It learns a hidden graph of dependencies (via an adaptive adjacency matrix) and captures long-range temporal trends via dilated convolutions. Tested on traffic speed networks (METR-LA, PEMS-BAY), GraphWaveNet outperforms prior baselines. Strengths: learns latent topology and long horizons end-to-end; code is open-source【74†L129-L134】. Weaknesses: requires a graph of nodes and enough data; not physics-aware (edge weights are learned). Datasets: public traffic sensors. Baselines: prior graph CNN/RNN models. *Reproducibility:* Official code on GitHub (cited by authors)【74†L129-L134】. **Key idea:** adaptive GNN+CNN for sequences【37†L31-L39】.

- **Diffusion Convolutional RNN (Li *et al.*, ICLR 2018)** – Models traffic flow as a *diffusion process on a directed graph*. It uses a *bidirectional diffusion convolution* (a graph convolution based on random walks) integrated into an RNN encoder-decoder with scheduled sampling【44†L23-L31】. Strengths: explicitly captures spatial/temporal dependencies with traffic analogy; shows ~12–15% improvement over strong baselines on large road networks【44†L23-L31】. Weaknesses: uses a fixed adjacency from road layout (no adaptivity); RNNs can struggle on very long sequences. Datasets: METR-LA, PEMS (traffic). Baselines: historical averaging, ARIMA, LSTMs, STGCN. *Reproducibility:* Authors released code (TensorFlow) and data. **Relevance:** Effective for any spatial network with flow (could map buses and lines, but must encode electrical connectivity).

- **Temporal Fusion Transformer (Lim *et al.*, ICLR 2021)** – A hybrid RNN+Transformer for **multi-horizon forecasting**【47†L23-L31】. It combines recurrent layers (for local context) with self-attention (for long-term patterns), plus gating/feature-selection layers. Designed to incorporate static covariates, known future inputs, etc. Strengths: Handles mixed inputs, provides interpretability (attention, gating)【47†L23-L31】. Weaknesses: Complex with many components, requires careful tuning; heavier training cost. Datasets: various real-world multivariate time series (e.g. traffic, electricity demand). Baselines: DeepAR, LSTM, TCN. *Reproducibility:* Open-source implementations exist (e.g. PyTorch Forecasting). **Note:** Can ingest exogenous features (e.g. weather, schedules) which may benefit load forecasting.

- **N-BEATS (Oreshkin *et al.*, NeurIPS 2020)** – A purely deep MLP with *backcast/forecast residual blocks* for univariate forecasting【58†L24-L32】. Remarkably, N-BEATS is *domain-agnostic*: with no specialized time-series layers, it sets new state-of-the-art on the M3/M4 benchmarks (~11% better than statistical baseline, ~3% better than previous winners)【58†L24-L32】. Strengths: simple architecture, interpretable basis expansion, fast training, top accuracy on diverse series. Weaknesses: only univariate (one series at a time), no explicit spatial context. Datasets: M3/M4 competition (economic/time series); authors also mention tourism. Baselines: classical ETS, ARIMA, hybrids, LSTM. *Reproducibility:* Official code (Element AI) is available. **Use case:** As a per-node forecasting baseline (e.g. predict load/voltage at each bus), especially when spatial links are weak or as an ablation.

- **Physics-Informed Neural Networks (PINNs)** – A paradigm (e.g. Raissi *et al.*, JCP 2019) for embedding physical laws into NNs【69†L142-L150】. A PINN is trained to satisfy known PDEs (e.g. conservation laws) as soft constraints, so the NN approximates a solution that obeys physics. Strengths: enforces consistency (e.g. Kirchhoff's laws, power-flow equations) and can improve generalization with limited data【69†L142-L150】. Weaknesses: requires exact knowledge of equations and boundary conditions; training can be difficult; relatively few power-grid applications yet. Datasets: usually synthetic PDE solutions or sensor data with known physics. Baselines: traditional numerical solvers; standard NNs. *Reproducibility:* Several frameworks (e.g. DeepXDE) exist. **Relevance:** A PINN could, for example, learn voltage profiles while exactly respecting power-flow equations, offering a hybrid physics+ML approach.

(Other relevant methods include general GNNs like GAT/GCN, autoencoders and GANs for anomaly detection, etc., though direct top-conference papers on those for power grids were not identified in open sources.)

Applicability to EnergyMind Challenge

- **Spatial-Temporal GNNs (GraphWaveNet, DCRNN):** These models assume a known graph of sensors (nodes) and learn dynamics over it. For power systems, nodes would be buses or PMU locations and edges power lines. They could forecast bus voltages or flows. *Adaptations:* Use the electrical admittance matrix for initial graph, or let the model adapt it【37†L31-L39】. Incorporate power-flow constraints post-hoc or via custom loss. *Data needs:* Historical time-series of system states (voltage phasors, flows) under varying conditions. IEEE test cases or real PMU data can be used. *Compute:* Training is moderate to heavy (GraphWaveNet reports fast training, and code is optimized). *Strengths:* Captures correlations across lines (e.g. a disturbance on one line affecting others). *Weaknesses:* Can be fooled by adversarial sensor attacks (since learned dependencies may

not enforce physics). Also, if topology changes (line faults), static-trained models may degrade. *Risk:* Must guard against false-data injection; robust training or detection needed.

- **Transformer/LSTM Forecasters (TFT, N-BEATS):** These models treat each time series (e.g. load at a bus, net injection) largely independently or with provided covariates. *Use case:* Short-term load/meter forecasting, which is part of resilience (predict demand, PV output, etc.). *Adaptations:* Include grid context by adding graph features as extra inputs, or run one model per group of buses. *Data needs:* Time-stamped measurements (with any known exogenous variables, e.g. weather, time-of-day). *Compute:* Transformers can be expensive but manageable for moderate horizons; N-BEATS is fast. *Strengths:* Proven accuracy on forecasting benchmarks; can handle multivariate inputs (TFT) or long-term trends. *Weaknesses:* Lack explicit power-flow physics or topology. *Risk:* Forecast errors could cascade into incorrect control actions; need uncertainty estimation.

- **Physics-Informed Models (PINNs):** *Applicability:* Enforcing grid equations (power-flow, Kirchhoff laws) during learning could greatly improve the plausibility of estimates (e.g. estimated states always satisfy load–flow balance). *Adaptations:* Formulate power-flow PDEs (actually algebraic equations) as loss constraints in the NN. Combine with measured data for semi-supervised learning. *Data needs:* Load/generation values, line parameters; perhaps synthetic grid states. *Compute:* Training PINNs is usually slower than black-box NNs due to the physics terms, but still feasible for medium-size grids. *Strengths:* Robust to noisy/partial data (physical prior); harder for an attacker to fool without violating laws. *Weaknesses:* Setting up the equations and tuning can be complex; performance depends on accuracy of model physics. *Risk:* Over-constraining could degrade if physics model is mismatched to reality.

- **Hybrid Classical-ML Pipelines:** A pragmatic baseline is a traditional *weighted least-squares (WLS) state estimator* (using SCADA/PMU data) combined with an ML-based anomaly detector. For example, run WLS to estimate bus voltages, then feed the residuals or estimated states into an autoencoder or classifier to flag anomalies/attacks. This leverages known physics (state estimation) while using ML to detect subtle patterns in the residual. *Pros:* Well-understood, interpretable SE. *Cons:* WLS can itself be misled by coordinated FDI attacks; ML requires good labeled anomalous data. *Compute:* Low for WLS; ML depends on model. *Data:* Power-flow models and injection measurements; plus simulated attack scenarios for training the detector.

- **Anomaly Detection (Graph Deviation, etc.):** Though not a single cited paper here, graph-based anomaly detectors like Graph Deviation Networks (GDN) have been proposed for industrial time-series. These learn a graph structure of sensor relationships and detect deviations. *Fit:* Could apply to PMU streams to localize attacks. *Adaptations:* Replace traffic sensors with PMUs, use power system adjacency. *Data:* Normal vs. fault/attack labeled sequences. *Risk:* May detect anomalies generally but not identify underlying faults without physics.

In summary, **Graph-based temporal models** (GraphWaveNet, DCRNN) are directly applicable to grid forecasting/anomaly detection if provided with the grid graph and time-series data【37†L31-L39】【44†L23-L31】. **Pure ML forecasters** (TFT, N-BEATS) are strong prediction baselines that may require feature engineering for power (e.g. encode network structure manually). **Physics-informed methods** could enhance trustworthiness by respecting power-flow equations【69†L142-L150】, but they need careful formulation. All approaches must be adapted to work with realistic simulator data and to handle adversarial scenarios (e.g. incorporating robust training or detection).

Baseline Pipeline Proposals

Below are 4 example end-to-end baseline pipelines combining physics-based and ML components:

Pipeline	Input & Preprocessing	Model(s)	Training Data	Eval Metrics	Pros	Cons
A: Classic SE + ML detector	SCADA/PMU measurements (voltages, injections). Run WLS state estimation using grid model.	*Physics:* Weighted least-squares state estimator; *ML:* autoencoder or random forest on residuals.	Simulated normal and fault/attack scenarios via MATPOWER/ OpenDSS.	State-estimation error; anomaly detection F1-score.	Uses known physics; fast & interpretable.	SE can be fooled by stealth attacks; detector needs labeled anomalies.
B: Graph Neural Forecaster	Time-series at each bus/ node, plus graph (line) connectivity (adjacency from admittances).	*ML:* Spatio-temporal GNN (e.g. Graph WaveNet or DCRNN) to predict next-state (voltages/ flows).	Historical grid time-series (real or simulated) covering normal and fault events.	Forecast MSE; anomaly detection via deviation threshold.	Learns complex correlations automatically 【37†L31-L39】.	Data-intensive; may not guarantee physical consistency.
C: Transformer Time-Series	Individual bus measurements (voltage/flow) plus exogenous covariates (weather, time).	*ML:* Temporal Fusion Transformer or LSTM ensemble to forecast each series jointly 【47†L23-L31】.	Multi-variate load/ injection timeseries with features.	Forecast accuracy (MAE); anomalies by large forecast errors.	Handles external features and long horizons; interpretable attention.	Large model size; ignores spatial links unless encoded.

Pipeline	Input & Preprocessing	Model(s)	Training Data	Eval Metrics	Pros	Cons
D: Physics-Informed NN	Raw sensor data and known grid equations.	*Hybrid:* PINN (or constrained GNN) trained with physics loss (power-flow equations) plus data loss.	Synthetic or historical data with known true states.	Physical violation error (e.g. power-balance residual); state prediction error.	Enforces Kirchhoff's laws 【69†L142-L150】; improved generalization.	Complex training; may need exact models; slower.

Each pipeline trades off "physics" vs. "learning". Pipeline A is safest but simpler; B/C can capture richer dynamics; D enforces consistency. The choice depends on data availability (e.g. actual PMU vs. simulated data) and compute budget. Evaluation metrics should include state estimation error, attack detection rate (precision/recall), forecasting RMSE, and timeliness of detection under attack.

Phase 1 Prototype Plan and Roadmap

1. **Milestones:**

2. **Data & Simulators (Weeks 1–4):** Acquire power system models (e.g. IEEE 14/57/118-bus test cases, distribution feeders). Set up simulators (MATPOWER, PandaPower, OpenDSS) to generate normal and fault scenarios (line outages, FDI attacks). Gather any available real PMU datasets.

3. **Baseline Implementation (Weeks 5–8):** Implement classical state estimator (WLS) and a simple residual detector. Build initial ML baselines: e.g. train GraphWaveNet and N-BEATS on normal operation data. Use publicly available tools (PyTorch, DGL, PyTorch Forecasting).
4. **Fault Injection & Testing (Weeks 9–12):** Systematically inject anomalies (e.g. sensor faults, FDI) and evaluate pipelines. Compare detection rates. Refine models (hyperparameters, input features).

5. **Integration & Evaluation (Weeks 13–16):** Combine best-performing components into a prototype pipeline. Establish evaluation protocol (hold-out scenario testing, cross-validation). Produce final performance summary.

6. **Data/Simulator Needs:**

7. **Network models:** IEEE standard grids, and Tunisian grid topology if available.

8. **Tools:** MATPOWER (for transmission PF), OpenDSS or GridLAB-D (for distribution/PV), Pandapower (Python-friendly).

9. **Fault injection tools:** Custom scripts to simulate FDI (adding realistic noise/spoofing to measurements) and contingencies. Use existing libraries or generate adversarial examples.

10. **Team Roles:**

11. **Power Systems Expert:** Defines realistic scenarios, validates physics, interprets results.

12. **Data Engineer:** Builds simulators, generates/cleans data, handles baseload data pipelines.
13. **ML Engineer/Researcher:** Implements and tunes ML models (GNNs, Transformers, PINNs).

14. **Dev/Ops & Visualization:** Integrates components, sets up training pipelines, develops evaluation dashboards.

15. **Evaluation Protocol:**

16. Use a mix of **regression metrics** (MSE/MAE for forecasts, state-estimation error) and **classification metrics** (accuracy, precision, recall for anomaly detection).

17. Include stress tests: e.g. adversarial scenarios, out-of-distribution loads.
18. Time-based metrics for resilience (time-to-detect).
19. Possibly human-in-the-loop evaluation if including "collective intelligence" (e.g. feedback on alerts).

(Throughout Phase 1, emphasize reproducibility: use version control, document code/data, and select baselines with available code and data whenever possible【74†L129-L134】【58†L24-L32】.)

Prioritized Implementation Roadmap

Based on the above, we recommend implementing these methods first (due to expected impact and available resources):

1. **Graph WaveNet & DCRNN** – Top priority. Both directly model spatio-temporal dynamics on graphs 【37†L31-L39】【44†L23-L31】. Code and benchmarks exist, so they provide strong baselines for forecasting state evolution (voltage/flows) and detecting anomalies via forecast residuals.
2. **N-BEATS (NeurIPS 2020)** – As a per-node forecasting baseline: it's state-of-art for single-series and very fast to train【58†L24-L32】. Useful as a lower-complexity comparison.
3. **Temporal Fusion Transformer** – For scenarios with rich covariates (e.g. weather for PV forecasting). It can improve multi-series forecasts once basic pipelines are in place【47†L23-L31】.
4. **Physics-based Estimator** – Implement a classical WLS state estimator early, as it provides a sanity-check and fallback. Then explore PINN-based augmentation.
5. **Anomaly Detection (GNN autoencoder)** – After establishing forecasting pipelines, experiment with dedicated anomaly detectors (e.g. autoencoders on state vectors) to catch FDI.

Each chosen method has published code or references to build on (e.g. GraphWaveNet's GitHub【74†L129-L134】), speeding up prototyping. We will iterate: start with simpler/fast-to-train models (N-BEATS, basic SE) to get a working demo, then layer on more complex hybrids.

Sources: We have drawn on primary conference papers and official sources, e.g. Wu *et al.* (IJCAI 2019) for GraphWaveNet【37†L31-L39】, Li *et al.* (ICLR 2018) for DCRNN【44†L23-L31】, Lim *et al.* (ICLR 2021) for TFT 【47†L23-L31】, Oreshkin *et al.* (NeurIPS 2020) for N-BEATS【58†L24-L32】, and Raissi *et al.* (JCP 2019) /

Wikipedia for PINNs【69†L142-L150】. Each cited paper includes method details, baselines, and (where available) links to code or data.
